

Module 7.3: Custom Pipes

Learning Objectives:

- Create custom pipes using Angular CLI and manual methods
 - Understand the @Pipe decorator and its configuration options
 - Implement the PipeTransform interface correctly
 - Build parameterized pipes that accept arguments
 - Understand pure vs impure pipes in custom implementations
-

1. Why Create Custom Pipes?

While Angular provides many built-in pipes, you'll often need custom transformations specific to your application:

- **Business-specific formatting:** Phone numbers, product codes, account numbers
 - **Domain-specific transformations:** Temperature conversion, file size formatting
 - **Custom text processing:** Truncation with ellipsis, highlighting, censoring
 - **Specialized calculations:** Time differences, distance formatting
-

2. Creating a Custom Pipe

2.1 Using Angular CLI (Recommended)

The easiest way to create a custom pipe is using the Angular CLI:

```
ng generate pipe reverse-text
```

Or using the shorthand:

```
ng g pipe reverse-text
```

This creates two files:

- `reverse-text.pipe.ts` - The pipe implementation
- `reverse-text.pipe.spec.ts` - The test file

And automatically updates your module's declarations.

2.2 Manual Creation

You can also create pipes manually. Here's what the CLI generates:

```
// reverse-text.pipe.ts
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'reverseText'
})
export class ReverseTextPipe implements PipeTransform {
  transform(value: string): string {
    // Implementation here
    return value;
  }
}
```

If creating manually, you must also add the pipe to your module's `declarations` array:

```
// app.module.ts
import { ReverseTextPipe } from './pipes/reverse-text.pipe';

@NgModule({
  declarations: [
    AppComponent,
    ReverseTextPipe // Add your pipe here
  ],
  // ...
})
```

3. The @Pipe Decorator

The `@Pipe` decorator configures your custom pipe:

```
@Pipe({
  name: 'reverseText',    // Name used in templates
  pure: true              // Optional: true by default
})
```

Property	Description	Default
<code>name</code>	The name used in templates (required)	-
<code>pure</code>	Whether the pipe is pure or impure	<code>true</code>

4. The PipeTransform Interface

All pipes must implement the `PipeTransform` interface, which requires a `transform` method:

```
export interface PipeTransform {
  transform(value: any, ...args: any[]): any;
}
```

The `transform` method:

- **First parameter:** The value to transform
 - **Additional parameters:** Optional arguments passed from the template
 - **Returns:** The transformed value
-

5. Simple Custom Pipe Example

Let's create a pipe that reverses text:

```
// reverse-text.pipe.ts
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'reverseText'
})
export class ReverseTextPipe implements PipeTransform {
  transform(value: string): string {
    if (!value) return value;
    return value.split('').reverse().join('');
  }
}
```

Component TypeScript:

```
export class AppComponent {
  message: string = 'Hello Angular';
  word: string = 'TypeScript';
}
```

Component Template:

```
<p>Original: {{ message }}</p>
<p>Reversed: {{ message | reverseText }}</p>
<p>Word: {{ word | reverseText }}</p>
```

Output:

Original: Hello Angular
Reversed: ralugnA olleH
Word: tpircSepyT

6. Pipes with Parameters

6.1 Single Parameter

Let's create a pipe that truncates text to a specified length:

```
// truncate.pipe.ts
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'truncate'
})
export class TruncatePipe implements PipeTransform {
  transform(value: string, limit: number = 50): string {
    if (!value) return value;

    if (value.length <= limit) {
      return value;
    }

    return value.substring(0, limit) + '...';
  }
}
```

Component TypeScript:

```
export class AppComponent {
  longText: string = 'Angular is a platform and framework for building single-page client
  applications using HTML and TypeScript.';
}
```

Component Template:

```
<p>Default (50 chars): {{ longText | truncate }}</p>
<p>30 characters: {{ longText | truncate:30 }}</p>
<p>70 characters: {{ longText | truncate:70 }}</p>
```

Output:

Default (50 chars): Angular is a platform and framework for building...

30 characters: Angular is a platform and fra...

70 characters: Angular is a platform and framework for building single-page client...

6.2 Multiple Parameters

Let's enhance the truncate pipe with a custom suffix:

```
// truncate.pipe.ts
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'truncate'
})
export class TruncatePipe implements PipeTransform {
  transform(value: string, limit: number = 50, suffix: string = '...'): string {
    if (!value) return value;

    if (value.length <= limit) {
      return value;
    }

    return value.substring(0, limit) + suffix;
  }
}
```

Component Template:

```
<p>Default suffix: {{ longText | truncate:40 }}</p>
<p>Custom suffix: {{ longText | truncate:40:' [more]' }}</p>
<p>Arrow suffix: {{ longText | truncate:40:' →' }}</p>
```

Output:

Default suffix: Angular is a platform and framework...

Custom suffix: Angular is a platform and framework [more]

Arrow suffix: Angular is a platform and framework →

7. More Custom Pipe Examples

7.1 File Size Formatter

```
// file-size.pipe.ts
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'fileSize'
})
export class FileSizePipe implements PipeTransform {
  transform(bytes: number, decimals: number = 2): string {
    if (bytes === 0) return '0 Bytes';

    const k = 1024;
    const sizes = ['Bytes', 'KB', 'MB', 'GB', 'TB'];
    const i = Math.floor(Math.log(bytes) / Math.log(k));

    return parseFloat((bytes / Math.pow(k, i)).toFixed(decimals)) + ' ' + sizes[i];
  }
}
```

Usage:

```
<p>Small file: {{ 1024 | fileSize }}</p>
<p>Medium file: {{ 1048576 | fileSize }}</p>
<p>Large file: {{ 1073741824 | fileSize }}</p>
```

Output:

Small file: 1.00 KB
Medium file: 1.00 MB
Large file: 1.00 GB

7.2 Phone Number Formatter

```
// phone.pipe.ts
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'phone'
})
export class PhonePipe implements PipeTransform {
  transform(value: string): string {
    if (!value) return value;

    // Remove all non-digit characters
    const cleaned = value.replace(/\D/g, '');

    // Format as (XXX) XXX-XXXX
    if (cleaned.length === 10) {
      return `(${cleaned.substring(0, 3)}) ${cleaned.substring(3, 6)}-${cleaned.substring(6)}`;
    }

    return value; // Return original if not 10 digits
  }
}
```

Usage:

```
<p>Phone 1: {{ '1234567890' | phone }}</p>
<p>Phone 2: {{ '555-867-5309' | phone }}</p>
```

Output:

Phone 1: (123) 456-7890
Phone 2: (555) 867-5309

7.3 Capitalize First Letter

```
// capitalize-first.pipe.ts
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'capitalizeFirst'
})
export class CapitalizeFirstPipe implements PipeTransform {
  transform(value: string): string {
    if (!value) return value;
    return value.charAt(0).toUpperCase() + value.slice(1).toLowerCase();
  }
}
```

Usage:

```
<p>{{ 'hello world' | capitalizeFirst }}</p>
<p>{{ 'ANGULAR PIPES' | capitalizeFirst }}</p>
```

Output:

```
Hello world
Angular pipes
```

8. Pure vs Impure Custom Pipes

8.1 Pure Pipes (Default)

Pure pipes are the default and recommended choice:

```
@Pipe({
  name: 'pureExample',
  pure: true // This is the default, can be omitted
})
export class PureExamplePipe implements PipeTransform {
```

```
transform(value: any): any {  
  console.log('Pure pipe executed');  
  return value;  
}  
}
```

Characteristics:

- Executes only when input value or parameters change
- Better performance
- Predictable behavior
- Recommended for most use cases

8.2 Impure Pipes

Use impure pipes when you need to detect changes within objects or arrays:

```
@Pipe({  
  name: 'impureExample',  
  pure: false // Makes the pipe impure  
})  
export class ImpureExamplePipe implements PipeTransform {  
  transform(value: any): any {  
    console.log('Impure pipe executed');  
    return value;  
  }  
}
```

When to use impure pipes:

- Filtering or sorting arrays that change internally
- Transformation depends on external state
- Need to detect property changes within objects

⚠ Warning:

Performance Impact: Impure pipes execute on every change detection cycle, which can significantly impact performance. Use them sparingly and only when necessary.

8.3 Example: Filter Pipe (Impure)

```
// filter.pipe.ts
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'filter',
  pure: false // Impure to detect array changes
})
export class FilterPipe implements PipeTransform {
  transform(items: any[], searchText: string, property: string): any[] {
    if (!items || !searchText) {
      return items;
    }

    searchText = searchText.toLowerCase();

    return items.filter(item => {
      return item[property].toLowerCase().includes(searchText);
    });
  }
}
```

Usage:

```
<input [(ngModel)]="searchTerm" placeholder="Search...">

<ul>
  <li *ngFor="let user of users | filter:searchTerm:'name'">
    {{ user.name }}
  </li>
</ul>
```

Tip:

Better Alternative: For filtering and sorting, consider implementing the logic in your component as a method or getter instead of using an impure pipe. This gives you more control and better performance.

9. Practical Example: Product Display

Let's combine multiple custom pipes in a product card:

Custom Pipes:

```
// discount.pipe.ts
@Pipe({ name: 'discount' })
export class DiscountPipe implements PipeTransform {
  transform(price: number, discountPercent: number): number {
    return price * (1 - discountPercent / 100);
  }
}

// stock-status.pipe.ts
@Pipe({ name: 'stockStatus' })
export class StockStatusPipe implements PipeTransform {
  transform(quantity: number): string {
    if (quantity === 0) return 'Out of Stock';
    if (quantity < 10) return 'Low Stock';
    return 'In Stock';
  }
}
```

Component:

```
export class ProductComponent {
  product = {
    name: 'wireless mouse',
    description: 'Ergonomic wireless mouse with USB receiver and long battery life',
    price: 29.99,
    discount: 15,
    stock: 8,
    sku: 'WM12345'
  };
}
```

Template:

```
<div class="product-card">
  <h2>{{ product.name | titlecase }}</h2>
  <p>{{ product.description | truncate:60 }}</p>
  <p>SKU: {{ product.sku | uppercase }}</p>

  <div class="price">
    <span class="original">{{ product.price | currency }}</span>
    <span class="discounted">
      {{ product.price | discount:product.discount | currency }}
    </span>
    <span class="save">Save {{ product.discount }}%</span>
  </div>

  <p class="stock">
    Status: {{ product.stock | stockStatus }}
    ({{ product.stock }} units available)
  </p>
</div>
```

Output:

Wireless Mouse

Ergonomic wireless mouse with USB receiver and long bat...

SKU: WM12345

\$29.99 **\$25.49** Save 15%

Status: Low Stock (8 units available)

10. Best Practices

Practice	Why It Matters
✓ Always implement PipeTransform	Ensures type safety and IDE support
✓ Handle null/undefined inputs	Prevents runtime errors

Practice	Why It Matters
✓ Use pure pipes by default	Better performance
✓ Keep transformation logic simple	Easier to test and maintain
✓ Use TypeScript types	Catches errors at compile time
✓ Provide default parameter values	Makes pipes easier to use
✓ Document complex pipes	Helps other developers

Common Mistakes to Avoid

✗ Mistake 1: Not handling null/undefined

```
// BAD
transform(value: string): string {
  return value.toUpperCase(); // Crashes if value is null
}

// GOOD
transform(value: string): string {
  if (!value) return value;
  return value.toUpperCase();
}
```

✗ Mistake 2: Using impure pipes unnecessarily

```
// BAD - Impacts performance
@Pipe({ name: 'uppercase', pure: false })

// GOOD - Pure by default
@Pipe({ name: 'uppercase' })
```

✗ Mistake 3: Mutating input values

```
// BAD - Mutates input array
transform(items: any[]): any[] {
  items.sort();
}
```

```
    return items;
  }

  // GOOD - Return new array
  transform(items: any[]): any[] {
    return [...items].sort();
  }
}
```

11. Summary

In this module, we covered custom pipe creation and advanced concepts:

Key Takeaways:

- **Creation Methods:** Use Angular CLI (`ng generate pipe`) or create manually
- **@Pipe Decorator:** Configures pipe name and pure/impure behavior
- **PipeTransform Interface:** Requires implementing the `transform()` method
- **Parameters:** Pipes can accept multiple parameters for customization
- **Pure vs Impure:**
 - Pure (default): Executes only when inputs change (better performance)
 - Impure: Executes on every change detection (use sparingly)
- **Best Practices:**
 - Always handle null/undefined inputs
 - Keep transformation logic simple
 - Use pure pipes by default
 - Don't mutate input values

Common Use Cases for Custom Pipes:

- Text formatting (truncation, phone numbers, capitalization)
- Data transformation (file sizes, discount calculation)
- Business-specific formatting (product codes, SKUs)
- Display enhancements (status badges, stock levels)

Next Steps: In the following module, we'll explore pipe chaining techniques and learn how to combine multiple pipes for complex data transformations.

12. Practice Exercises

Exercise 1: Credit Card Mask Pipe

Create a pipe that masks credit card numbers:

- Input: "1234567890123456"
- Output: "**** * 3456"
- Parameter: number of visible digits (default: 4)

Exercise 2: Pluralize Pipe

Create a pipe that pluralizes words based on count:

- Usage: `{{ count | pluralize:'item' }}`
- Output: "1 item" or "5 items"
- Handle irregular plurals: `{{ count | pluralize:'child':'children' }}`

Exercise 3: Temperature Converter Pipe

Create a pipe that converts temperatures:

- Usage: `{{ temp | temperature:'C' }}` (converts to Celsius)
- Usage: `{{ temp | temperature:'F' }}` (converts to Fahrenheit)
- Display with unit symbol

Exercise 4: Time Ago Pipe

Create a pipe that shows relative time:

- Input: Date object
- Output: "2 hours ago", "3 days ago", "1 month ago"

- Handle future dates: "in 2 hours"

Module 7.3 - Custom Pipes | Angular Framework Training

